OPT Project3
발표

<Group 3>

미래자동차공학과2019048440 최윤석

전기공학전공2020090846 김영민

발표 순서

- 서론: Project 3 개요
- Problem 1: MiniGrid 환경 구성
- Problem 2: Q-Learning & SARSA with Maze Problem
- Problem 3: DQN with Maze Problem

서론: Project 3 개요

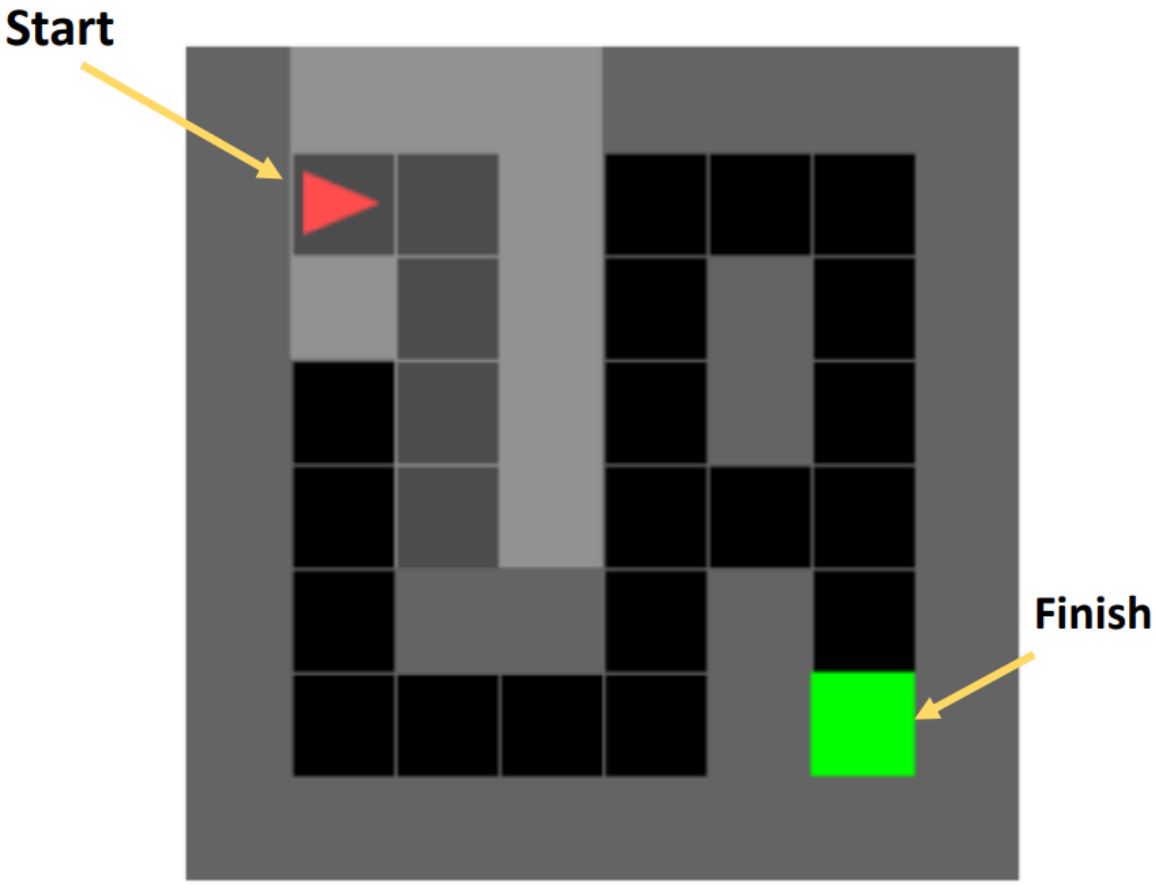
프로젝트 목적

강화학습을 활용한 미로 탐색 경로 학습 실험
실험 환경: MiniGrid

문제 구성

- ✓ Problem 1: 사용자 정의 미로 환경 구성
- ✓ Problem 2: Q-Learning vs. SARSA 적용
- ✓ Problem 3: DQN을 통한 성능 비교 분석

Problem 1: 개요



- ✓ **환경:** 6x6 격자 미로
- ✓ **에이전트 행동:** Left: 좌회전, Right: 우회전 ,Forward: 전진
- ✓ **출발 지점:** (1,1), **도착 지점:** (6,6)
- ✓ **제약 조건:** 회색 벽(Wall)으로 인한 경로 제한, 방향을 고려한 경로 탐색 필요
- ✓ **결론:** 단순 최단 거리 탐색 알고리즘으로는 불

Problem 1: Pseudo 코드

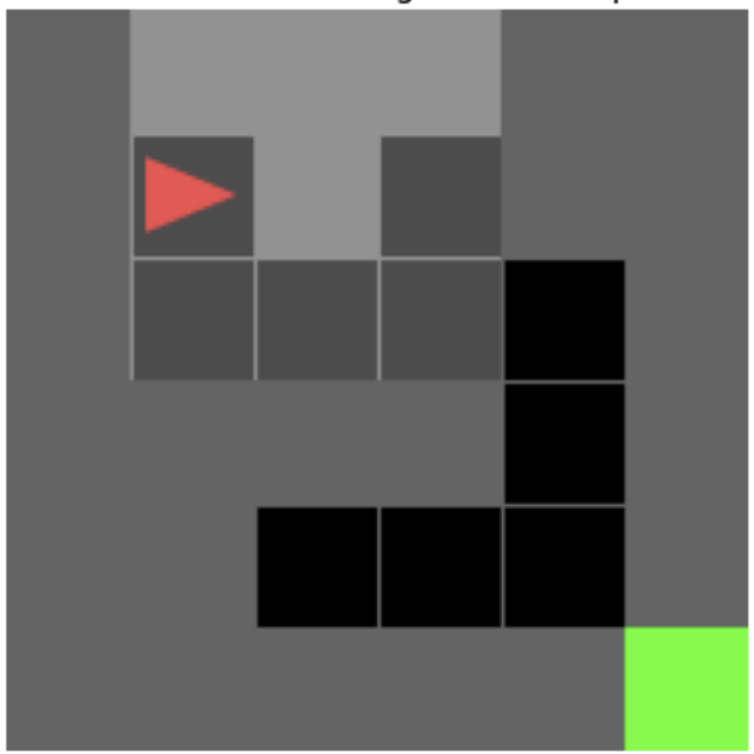
```
# 사용자 정의 6x6 미로 환경
class CustomMazeEnv(MiniGridEnv):
    def __init__(self, size=6, render_mode=None):
        self.size = size
        mission_space = Text(max_length=64)

        super().__init__(
            grid_size=size,
            max_steps=4 * size * size,
            agent_view_size=3,
            mission_space=mission_space,
            render_mode=render_mode
        )
```

```
    self.action_space = gym.spaces.Discrete(3)  # 좌, 우, 전진(Three Actions)

def _gen_grid(self, width, height):
    self.grid = Grid(width, height)
    self.grid.wall_rect(0, 0, width, height)
```

```
# 슬라이드 기반 벽 위치 (에이전트 위치와 겹치지 않도록 (1,1)은 제외)
wall_positions = [
    (1, 3), (1, 4),
    (2, 1), (2, 3),
    (3, 3),
    (4, 1), (4, 5),
    (5, 5),
]
for pos in wall_positions:
    self.put_obj(Wall(), *pos)

# 종료 지점 (6,6) → 인덱스 기준 (5,5)
self.put_obj(Goal(), 5, 5)

# 시작 위치 (1,1) → 인덱스 기준 (1,1) ← (1,1)이 이제 비어 있음
self.agent_pos = (1, 1)
self.agent_dir = 0
self.mission = "Reach the green goal square"
```

```
# 환경 생성 및 리셋
env = CustomMazeEnv(render_mode="rgb_array")
obs, _ = env.reset()

# 이미지 렌더링
image = env.render()

# 출력
plt.imshow(image)
plt.axis("off")
plt.title("6x6 Maze Matching Slide Ex
plt.show()
```

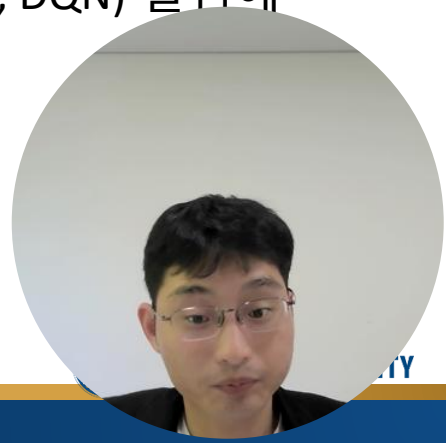
Problem 1: 코드 실행 결과



✓ **환경 시각화 결과**

- 에이전트의 시작 위치 및 목표 위치 명확
- 벽(Wall)과 통로(Aisle)의 배치가 정확하게 구현됨
- 실제 시뮬레이션에서 정상 동작 확인

☞ 강화학습 알고리즘 (Q-learning, SARSA, DQN) 실험에 적합한 테스트 환경 제공



Problem 2-1: 개요(Q-Learning)

- ✓ **Temporal Difference** 방식의 Value 기반 강화학습
- ✓ **탐험-이용** 균형을 위한 epsilon-greedy 정책 사용
- ✓ **벨만 최적 방정식**에 기반한 반복 갱신
- ✓ 각 상태-행동 쌍 (s, a)에 대해 다음 상태 s'에서 **가능한 Q값 중 MAX** 기반으로 업데이트

$$Q(s,a) \leftarrow Q(s,a) + \alpha \cdot \left[r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a) \right]$$

Problem 2-1: 벨만 최적 방정식(Q-Learning)

- ✓ **α 학습률**(learning rate) : 새로 관측된 정보가 기존 Q값에 얼마나 반영될지를 결정
- ✓ **γ 할인율**(discount factor) : 미래 보상의 현재 가치에 대한 중요도를 조절하는 계수
- ✓ 각 상태-행동 쌍 (s, a)에 대해 다음 상태 s'에서 **가능한 Q값 중 MAX** 기반으로 업데이트
- ✓ Q-Learning은 Off-Policy 방식

$$Q(s,a) \leftarrow Q(s,a) + \alpha \cdot \left[r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a) \right]$$

Problem 2-1: Q-Learning

```python
# ==================== 사용자 정의 환경 ====================
class SimpleEnv(MiniGridEnv):
    def __init__(self, size=6, agent_start_pos=(1, 1), agent_start_dir=0, **kwargs):
        self.agent_start_pos = agent_start_pos
        self.agent_start_dir = agent_start_dir

        mission_space = Text(max_length=64)
        super().__init__(
            grid_size=size,
            max_steps=4 * size * size,
            agent_view_size=3,
            mission_space=mission_space,
            **kwargs
        )

        self.action_space = gym.spaces.Discrete(3)  # Left, Right, Forward

    @staticmethod
    def _gen_mission():
        return "grand mission"

    def _gen_grid(self, width, height):
        self.grid = Grid(width, height)
        self.grid.wall_rect(0, 0, width, height)

        wall_positions = [
            (1, 3), (1, 4),
            (2, 1), (2, 3),
            (3, 3),
            (4, 1), (4, 5),
            (5, 5),
        ]
        for pos in wall_positions:
            self.put_obj(Wall(), *pos)

        self.put_obj(Goal(), width - 2, height - 2)  #  유효한 내부 위치

        self.agent_pos = self.agent_start_pos
        self.agent_dir = self.agent_start_dir
        self.mission = "Reach the green goal square"
```

- ✓ **환경:** 6x6 격자 미로
- ✓ **에이전트 행동:** Left: 좌회전, Right: 우회전 ,Forward: 전진
- ✓ **출발 지점:** (1,1), **도착 지점:** (6,6)
- ✓ **제약 조건:** 회색 벽(Wall)으로 인한 경로 제한, 방향을 고려한 경로 탐색 필요
- ✓ **결론:** 단순 최단 거리 탐색 알고리즘으로는 불충분

Problem 2-1: Q-Learning

Epsilon-Greedy

```
# ==================== Epsilon-Greedy ====================
def eps_greedy(Q_values, state, epsilon=1.0):
    if random.random() < epsilon:
        return random.randint(0, 2)  # 3개 행동 중 무작위
    else:
        return int(np.argmax(Q_values[state]))
```

- ✓ 탐험(exploration)-활용(exploitation) 전략
- ✓ 역할 : 현재 상태에서 어떤 행동을 선택할지
- ✓ Q_values : 학습된 상태-행동 가치 함수 (Q-table)
- ✓ State : 현재 상태(좌표, 방향 등)
- ✓ Epsilon : 무작위 탐험을 시도할 확률

Problem 2-1: Q-Learning

Epsilon-Greedy

```
# ==================== Epsilon-Greedy ====================
def eps_greedy(Q_values, state, epsilon=1.0):
    if random.random() < epsilon:
        return random.randint(0, 2)  # 3개 행동 중 무작위
    else:
        return int(np.argmax(Q_values[state]))
```

- ✓ Q-Learning은 보상을 최대화하는 방향으로 학습
- ✓ 초기에는 Q가 전부 0 → 어떤 행동이 좋은지 모름
- ✓ No Exploration → Q-table 갱신 X
- ✓ Always Exploration → 학습이 잘 안됨

- ✓ 둘을 절충하기 위한 **Epsilon-Greedy**
- ✓ **Epsilon**만큼 임의의 행동
- ✓ **1-Epsilon**만큼 Q값이 가장 높은 행동

Problem 2-1: Q-Learning

```python
# ==================== Q-Learning ====================
def run(env, Q_values, n_episodes, epsilon, epsilon_decay, gamma, lr):
    total_rewards = []

    for ep in tqdm(range(n_episodes)):
        obs, info = env.reset()
        state = (env.agent_pos[0], env.agent_pos[1], env.agent_dir)
        done = False
        ep_reward = 0

        while not done:
            action = eps_greedy(Q_values, state, epsilon)
            next_obs, reward, terminated, truncated, info = env.step(action)
            next_state = (env.agent_pos[0], env.agent_pos[1], env.agent_dir)
            done = terminated or truncated

            best_next_q = np.max(Q_values[next_state])
            td_target = reward + gamma * best_next_q
            td_error = td_target - Q_values[state][action]
            Q_values[state][action] += lr * td_error

            state = next_state
            ep_reward += reward

        epsilon *= epsilon_decay
        total_rewards.append(ep_reward)

    return Q_values, total_rewards
```

- ✓ 에이전트가 환경을 통해 최적 정책을 학습
- ✓ Q_table 업데이트하면서 최적의 상태-행동 가치 함수 Q(s, a)
- ✓ Off-policy TD 알고리즘

- ✓ 간단한 구조
- ✓ 충분한 에피소드 동안 수렴 보장
- ✓ 최적 행동 따라가지 않아도 학습 가능

Problem 2-1: Q-Learning

```python
# ==================== 학습 실행 ====================
env = SimpleEnv(render_mode="rgb_array")
Q_values = np.zeros((env.width, env.height, 4, env.action_space.n))  # (x, y, dir, action)

Q_values, rewards = run(
    env,
    Q_values,
    n_episodes=5000,
    epsilon=1.0,
    epsilon_decay=0.999,  #  더 천천히 감소
    gamma=0.99,
    lr=0.1
)
```

✓ 초기 탐험 위주 → Exploitation으로 전환

✓ Q-Learning 학습 실행

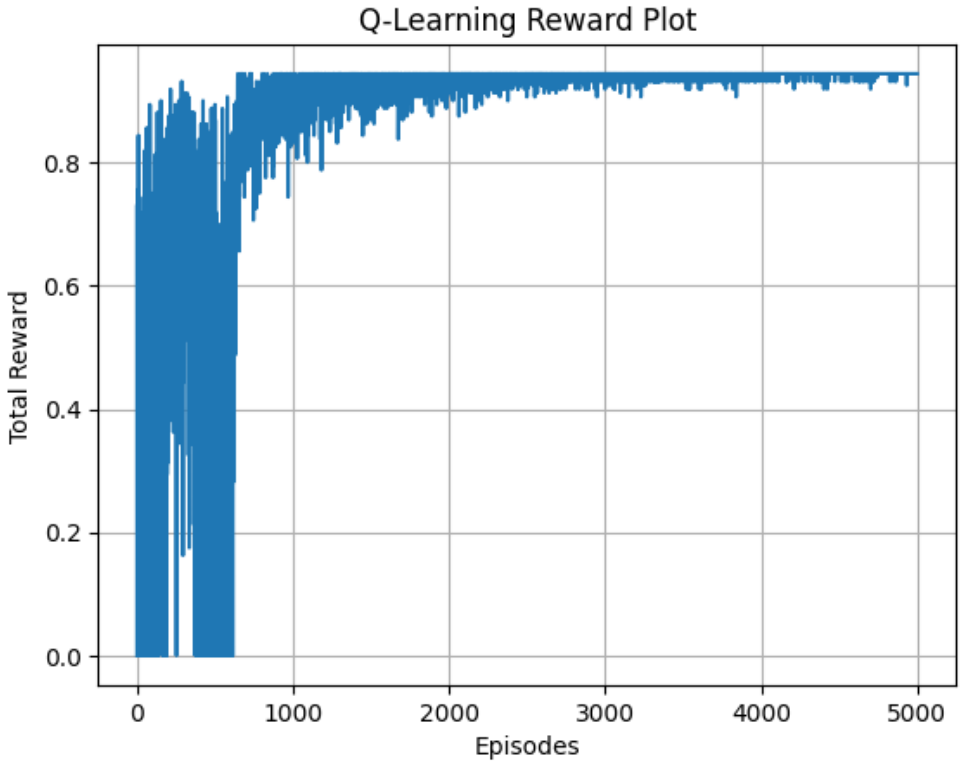
```python
# ==================== 학습된 정책 테스트 ====================
frames = []
obs, info = env.reset()
state = (env.agent_pos[0], env.agent_pos[1], env.agent_dir)
done = False
trajectory = [state]
frames.append(env.render())

while not done:
    action = np.argmax(Q_values[state])
    obs, reward, terminated, truncated, info = env.step(action)
    state = (env.agent_pos[0], env.agent_pos[1], env.agent_dir)
    trajectory.append(state)
    frames.append(env.render())
    done = terminated or truncated
```

✓ 학습된 Q-table 기반으로 행동 결정

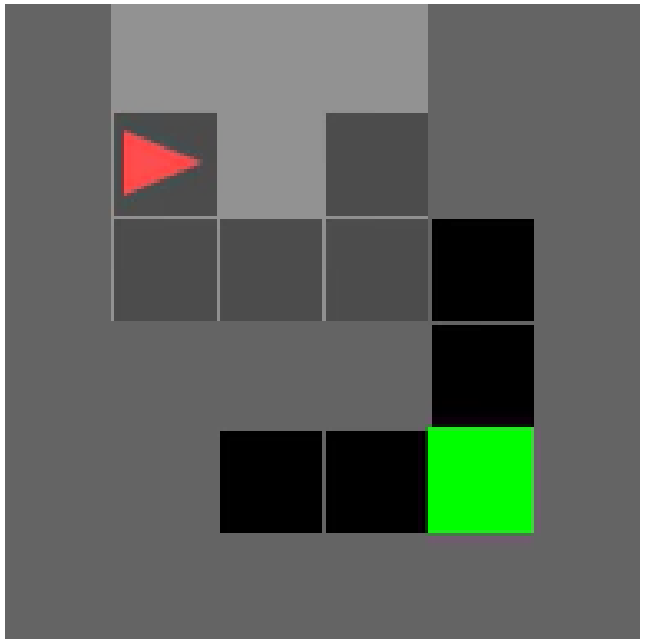
✓ **Successful?**

✓ 최단거리로 목적지 도달

Problem 2-1: Q-Learning



✅ State trajectory (x, y, direction):
(1, 1, 0)
(1, 1, 1)
(np.int64(1), np.int64(2), 1)
(np.int64(1), np.int64(2), 0)
(np.int64(2), np.int64(2), 0)
(np.int64(3), np.int64(2), 0)
(np.int64(4), np.int64(2), 0)
(np.int64(4), np.int64(2), 1)
(np.int64(4), np.int64(3), 1)
(np.int64(4), np.int64(4), 1)



✓ 초기 탐험 무작위 → 보상 변동 폭 큼
✓ 1000 이후에는 보상 1에 근접

Problem 2-2: SARSA

- ✓ **On-Policy TD 학습 알고리즘**
- ✓ 에이전트가 실제 취한 행동 기반으로 Q 갱신
- ✓ 현재 상태, 행동, 보상, 다음 상태, 행동 5가지 업데이트
- ✓ Q-Learning보다 보수적
- ✓ 안정적인 수렴 유도 유리

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \cdot [R_{t+1} + \gamma \cdot Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$$

Problem 2-2: SARSA

```python
# SARSA 학습 함수
def run_sarsa(env, Q_values, n_episodes, epsilon, epsilon_decay, gamma, lr):
    total_rewards = []
    for _ in tqdm(range(n_episodes)):
        obs, info = env.reset()
        state = (env.agent_pos[0], env.agent_pos[1], env.agent_dir)
        action = eps_greedy(Q_values, state, epsilon)
        done = False
        ep_reward = 0

        while not done:
            next_obs, reward, terminated, truncated, info = env.step(action)
            next_state = (env.agent_pos[0], env.agent_pos[1], env.agent_dir)
            done = terminated or truncated
            next_action = eps_greedy(Q_values, next_state, epsilon)

            # SARSA 업데이트
            td_target = reward + gamma * Q_values[next_state][next_action]
            td_error = td_target - Q_values[state][action]
            Q_values[state][action] += lr * td_error

            state = next_state
            action = next_action
            ep_reward += reward

        epsilon *= epsilon_decay
        total_rewards.append(ep_reward)
    return Q_values, total_rewards
```

- ✓ **On-Policy TD 학습 알고리즘**
- ✓ 에이전트가 실제 취한 행동 기반으로 Q 갱신
- ✓ 현재 행동 → 다음 상태 및 보상 관측
  (Q-Learning과의 차이)
- ✓ TD target : 다음 상태에서 실제 선택된 행동의 Q값 사용

  → Q-table 업데이트, 보수적이고 안정적인 수렴

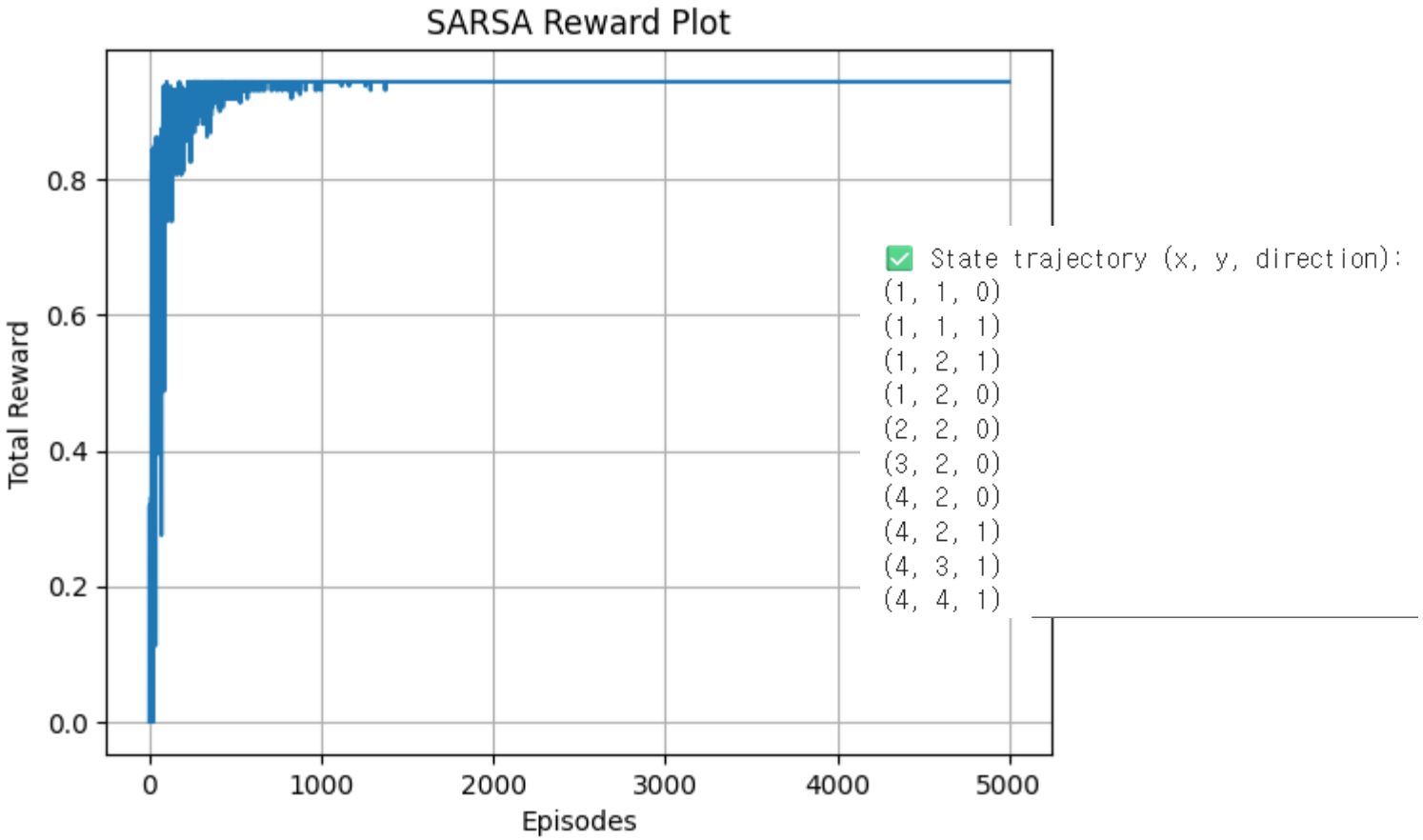
Problem 2-2: SARSA

```
# 환경 및 학습 실행
env = SimpleEnv(render_mode="rgb_array", agent_start_pos=(1, 1))
Q_values = np.zeros((env.width, env.height, 4, 3))

Q_values, rewards = run_sarsa(
    env,
    Q_values,
    n_episodes=5000,
    epsilon=1.0,
    epsilon_decay=0.995,
    gamma=0.99,
    lr=0.1
)
```

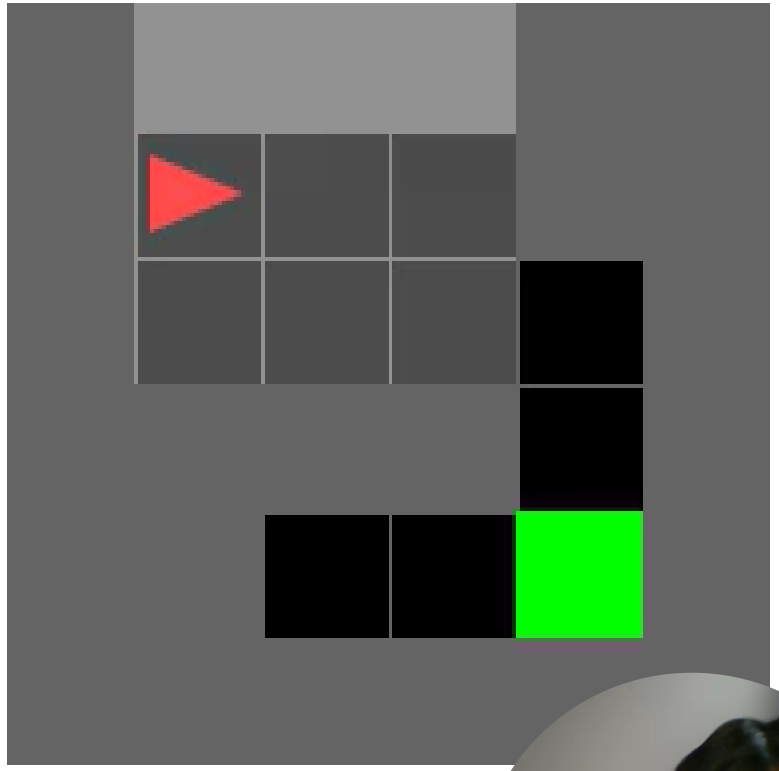
```
# 에이전트의 trajectory 수집 및 시각화
trajectory = []
frames = []
obs, info = env.reset()
state = (env.agent_pos[0], env.agent_pos[1], env.agent_dir)
trajectory.append(state)
frames.append(env.render())
done = False
```

- ✓ 에피소드 수 만큼 반복
- ✓ Epsilon-greedy 탐험/이용 정책 적용
- ✓ Q값 업데이트 (SARSA)

Problem 2-2: SARSA



State trajectory (x, y, direction):
(1, 1, 0)
(1, 1, 1)
(1, 2, 1)
(1, 2, 0)
(2, 2, 0)
(3, 2, 0)
(4, 2, 0)
(4, 2, 1)
(4, 3, 1)
(4, 4, 1)



Problem 3: DQN

- ✓ Q-Learning 의 Q-테이블을 **신경망**으로 대체
- ✓ 상태공간이 연속적이거나 고차원일때 유리
- ✓ **상태 s를 입력, 가능한 모든 행동 a에 대한 Q(s,a) 출력 (회귀 모델)**
- ✓ 손실함수를 최소화하며 학습 수행

$$\text{Loss} = \left[r + \gamma \cdot \max_{a'} Q_{\text{target}}(s', a') - Q(s, a) \right]^2$$

Problem 3: DQN

```python
# ------------------ 2. Replay Buffer ------------------
class ReplayBuffer:
    def __init__(self, capacity):
        self.buffer = deque(maxlen=capacity)
    def push(self, s, a, r, s_, d):
        self.buffer.append((s, a, r, s_, d))
    def sample(self, batch_size):
        s, a, r, s_, d = zip(*random.sample(self.buffer, batch_size))
        return s, a, r, s_, d
    def __len__(self):
        return len(self.buffer)
```

✓ **Replay Buffer** : 데이터 상관성 제거, 학습 효율 향상

✓ **과거의 경험을 저장, 무작위로 샘플링하여 학습에 사용**

Problem 3: DQN

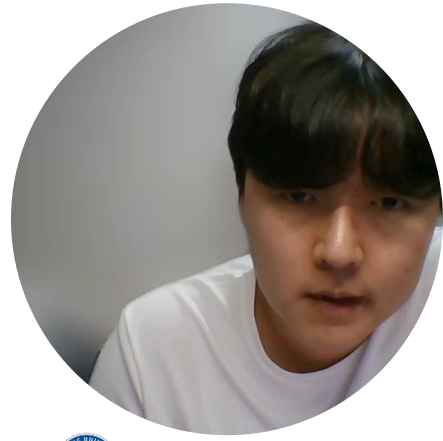
```
# ------------------ 3. DQN Model ------------------
class DQN(nn.Module):
    def __init__(self, input_shape, num_actions):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(input_shape[0], 16, 5, stride=1), nn.ReLU(),
            nn.Conv2d(16, 32, 3, stride=1), nn.ReLU(),
            nn.Conv2d(32, 32, 3, stride=1), nn.ReLU()
        )
        with torch.no_grad():
            dummy = torch.zeros(1, *input_shape)
            conv_out = self.conv(dummy)
            self.flat_dim = int(np.prod(conv_out.size()))
        self.fc = nn.Sequential(
            nn.Linear(self.flat_dim, 256), nn.ReLU(),
            nn.Linear(256, num_actions)
        )
    def forward(self, x):
        x = self.conv(x)
        x = x.view(x.size(0), -1)
        return self.fc(x)
```

- ✓ 입력 : 환경으로부터 받은 이미지
- ✓ 출력 : 가능한 모든 행동에 대한 Q값
- ✓ 입력 이미지 → CNN → Flatten → FC → Q값

Problem 3: DQN

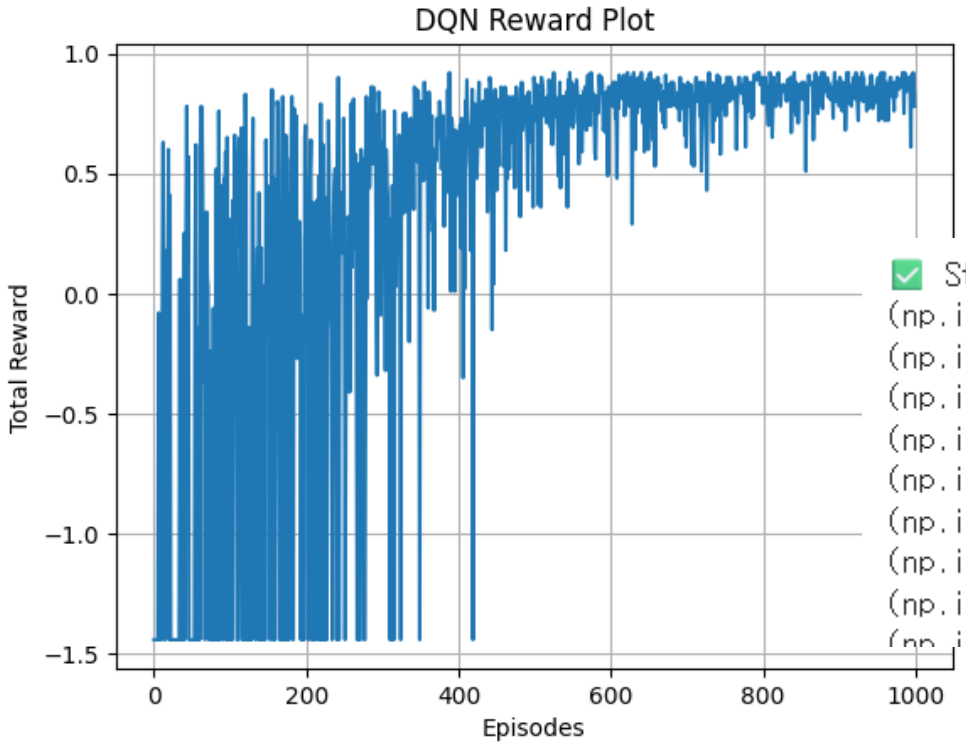
```python
# ------------------ 6. DQN 학습 루프 ------------------
def train_dqn(env, model, target_model, buffer, optimizer,
              episodes, batch_size, gamma, epsilon, epsilon_decay, update_freq=20):
    rewards = []
    for ep in tqdm(range(episodes)):
        obs, _ = env.reset()
        state = preprocess(obs).to(device)
        done, total_reward = False, 0

        while not done:
            action = select_action(model, state, epsilon, env.action_space.n)
            obs_, reward, terminated, truncated, _ = env.step(action)
            next_state = preprocess(obs_).to(device)
            done = terminated or truncated
            buffer.push(state, action, reward, next_state, done)
            state = next_state
            total_reward += reward

            if len(buffer) >= batch_size:
                b = buffer.sample(batch_size)
                s = torch.cat(b[0]).to(device)
                a = torch.tensor(b[1]).unsqueeze(1).to(device)
                r = torch.tensor(b[2], dtype=torch.float32).unsqueeze(1).to(device)
                s_ = torch.cat(b[3]).to(device)
                d = torch.tensor(b[4], dtype=torch.float32).unsqueeze(1).to(device)

                q = model(s).gather(1, a)
                with torch.no_grad():
                    max_q = target_model(s_).max(1)[0].unsqueeze(1)
                    target = r + gamma * max_q * (1 - d)

                loss = F.mse_loss(q, target)
                optimizer.zero_grad()
                loss.backward()
                optimizer.step()

        epsilon = max(epsilon * epsilon_decay, 0.01)
        rewards.append(total_reward)

        if ep % update_freq == 0:
            target_model.load_state_dict(model.state_dict())

    return rewards
```

- ✓ 에피소드 루프 시작
- ✓ 에이전트 동작
- ✓ 경험 저장 (Replay Buffer)
- ✓ Replay Buffer 에서 학습 수행
- ✓ 역전파 및 최적화
- ✓ Target Network 동기화

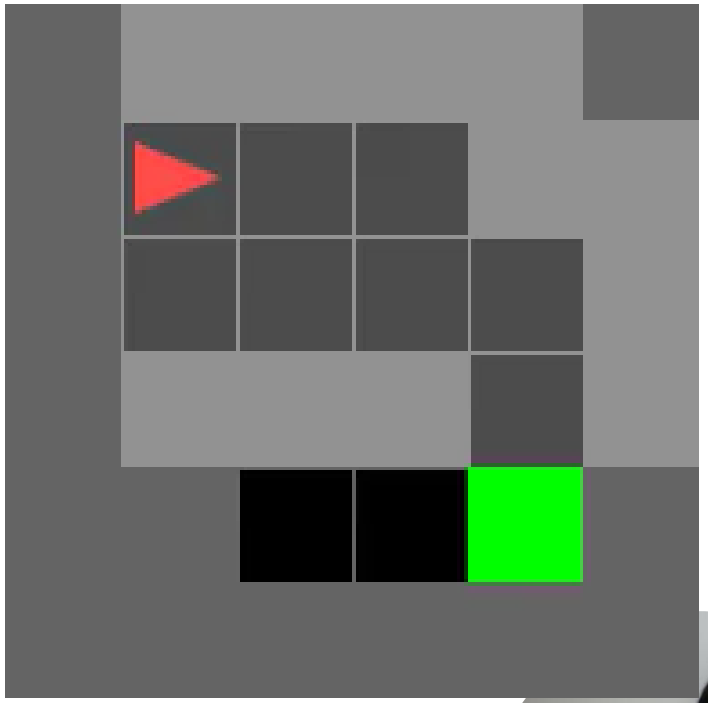


HANYANG UNIVERSITY

Problem 3: DQN



✅ State trajectory (x, y, direction):
(np.int64(2), np.int64(1), 0)
(np.int64(2), np.int64(1), 1)
(np.int64(2), np.int64(2), 1)
(np.int64(2), np.int64(2), 0)
(np.int64(3), np.int64(2), 0)
(np.int64(4), np.int64(2), 0)
(np.int64(4), np.int64(2), 1)
(np.int64(4), np.int64(3), 1)
(np.int64(4), np.int64(4), 1)



감사합니다!